SIMATS Engineering
Department of Information Technology
MLA0402 - DEEP LEARNING
ASSIGNMENT 1
CO1 – LEARNING ALGORITHMS, ESTIMATION AND NEURAL NETWORK FOUNDATIONS

Nature: Individual
Duration: 1 Week **Total Marks:** 50

Name: Karthik V - 192425071

CO1: Apply and analyze learning algorithms, maximum likelihood estimation, the process of building a machine learning algorithm, multilayer perceptrons, back-propagation with stochastic gradient descent and its variants, and the curse of dimensionality to make justified modelling decisions.

Topics: Learning Algorithms, Maximum Likelihood Estimation, Building a Machine Learning Algorithm, Neural Networks – Multilayer Perceptron, Back-propagation Algorithm and its Variants (Stochastic Gradient Descent), Curse of Dimensionality.

Purpose

Investigate how learning principles, estimation theory and optimization behaviour influence the reliability of neural-network-based systems. Students must connect equations, assumptions and numerical behaviour to practical model decisions.

Scenario: Smart Manufacturing Quality Inspection

A factory plans to build an intelligent system that predicts whether a manufactured component is defective, using sensor readings such as temperature, vibration, pressure and humidity recorded during production. The factory holds labelled inspection records, and it must decide which learning approach, estimation method, network design and training procedure to use. You are required to analyze the data, select suitable techniques and justify every decision using mathematical reasoning.

TASK 1 – Learning Algorithms: Task, Experience and Performance | 4 Marks

The factory has 5,000 past inspection records. Each record contains four sensor readings and a label stating whether the component was “defective” or “acceptable”. The system must label new components automatically.

a) Define the learning problem using Task (T), Experience (E) and Performance measure (P).

b) Identify the learning paradigm used and justify why a fixed hand-written rule is less suitable than a learning algorithm.

c) Explain why accuracy alone may be misleading if only 3% of components are defective, and name two better measures.

Evidence Expected: *T, E, P definition, identification of the learning paradigm, justification, and metric reasoning.*

TASK 2 – Capacity, Overfitting and Generalization | 3 Marks

Three candidate models are evaluated. Model X has a training error of 18% and a validation error of 20%. Model Y has a training error of 2% and a validation error of 19%. Model Z has a training error of 6% and a validation error of 8%.

a) Identify which model underfits, which overfits and which generalizes best.

b) Explain how model capacity and the generalization gap are related.

c) Suggest one suitable improvement each for Models X and Y.

Evidence Expected: *Correct identification, capacity explanation, and justified remedies.*

TASK 3 – Maximum Likelihood Estimation: Bernoulli Model | 4 Marks

In one production batch, 50 components are inspected and 7 are found defective. Let each component be defective with unknown probability p, independently of the others.

a) Write the likelihood and log-likelihood function for the observed data.

b) Derive the maximum likelihood estimate of p and compute its value.

c) Using your estimate, calculate the probability that the next three inspected components are all acceptable, and comment on the reliability of the estimate for a small sample.

Evidence Expected: *Likelihood expression, derivation steps, numerical results and interpretation.*

TASK 4 – Maximum Likelihood Estimation: Gaussian Model and Link to MSE | 5 Marks

A furnace temperature sensor records the readings 68, 70, 72, 74 and 76 °C, assumed to be drawn independently from a Gaussian distribution $N(\mu, \sigma^2)$.

a) Write the Gaussian likelihood and log-likelihood function.

b) Derive the MLE of μ and σ^2 and compute their values for the given readings.

c) Explain why the MLE of σ^2 is biased, compute the unbiased estimate using $n - 1$, and compare the two values.

d) For the regression model $y = wx + b + \varepsilon$ with $\varepsilon \sim N(0, \sigma^2)$, show that maximizing the likelihood is equivalent to minimizing the mean squared error.

Evidence Expected: *Derivations, numerical solutions, bias discussion and the MLE–MSE equivalence.*

TASK 5 – Building a Machine Learning Algorithm | 5 Marks

The factory wants to predict tool wear y (in units of 0.1 mm) from running time x (in hours). Four observations are available: (x, y) = (1, 3), (2, 5), (3, 6), (4, 9). The proposed model is $\hat{y} = wx + b$ with cost $J = (1/m) \Sigma (\hat{y} - y)^2$.

a) Identify the dataset, model, cost function and optimization procedure that together form this learning algorithm.

b) Derive the expressions for $\partial J/\partial w$ and $\partial J/\partial b$.

c) Starting from $w = 0$ and $b = 0$ with learning rate $\alpha = 0.05$, compute the gradients and perform one gradient descent update. Report the new w and b.

d) Differentiate between parameters and hyperparameters in this problem, and explain the effect of choosing α too large or too small.

Evidence Expected: *Four-component specification, gradient derivation, numerical update and hyperparameter reasoning.*

TASK 6 – Multilayer Perceptron: Architecture and Forward Pass | 4 Marks

A Multilayer Perceptron has 2 inputs, one hidden layer with 2 ReLU neurons, and 1 sigmoid output neuron. The input is x = (0.5, 1.0). Hidden weights and biases: h_1 uses weights (0.4, 0.2) and bias 0.1;

h_2 uses weights $(-0.6, 0.5)$ and bias -0.2. Output weights are $v_1 = 0.8$ and $v_2 = 0.3$ with output bias -0.1.

a) Draw the network and count the number of trainable parameters.

b) Perform the forward pass and compute h_1, h_2 and the output $\hat{y}$.

c) Explain why a hidden layer with a non-linear activation is necessary, and what happens if all activations are linear.

Evidence Expected: *Network diagram, parameter count, forward-pass calculations and explanation of non-linearity.*

TASK 7 – Back-propagation Algorithm | 5 Marks

Use the network and input of Task 6 with target $t = 1$ and loss $L = \frac{1}{2}(t - \hat{y})^2$. The output neuron is sigmoid and the hidden neurons are ReLU.

a) Write the chain-rule expressions for $\partial L/\partial v_1$ and $\partial L/\partial w_{11}$, where w_{11} is the weight from the first input to h_1.

b) Compute $\partial L/\partial v_1$, $\partial L/\partial v_2$, $\partial L/\partial$(output bias) and $\partial L/\partial w_{11}$ numerically.

c) Update v_1 and w_{11} using learning rate $\eta = 0.1$.

d) Explain why back-propagation is more efficient than computing every derivative independently.

Evidence Expected: *Chain-rule derivation, step-by-step numerical gradients, weight update and efficiency explanation.*

TASK 8 – Vanishing Gradient and Activation Functions | 3 Marks

The factory deepens its network to 10 hidden layers, all using sigmoid activation. Training loss barely decreases in the early layers.

a) Show that the derivative of the sigmoid is at most 0.25 and calculate the upper bound 0.25^{10} on the product of activation derivatives across 10 layers.

b) Explain how this causes vanishing gradients and what it does to learning in the early layers.

c) Suggest two remedies and justify each.

Evidence Expected: *Derivative bound, numerical computation, explanation of the effect and justified remedies.*

TASK 9 – Batch, Stochastic and Mini-batch Gradient Descent | 3 Marks

The factory trains on $N = 1{,}000{,}000$ inspection records and compares batch gradient descent, stochastic gradient descent (SGD) and mini-batch gradient descent with a batch size of 100.

a) Calculate the number of parameter updates in one epoch for each method.

b) Compare the methods in terms of gradient noise, memory use, cost per update and convergence behaviour.

c) Recommend one method for this dataset and justify the choice.

Evidence Expected: *Update counts, comparison of the three methods and a justified recommendation.*

TASK 10 – SGD Variants: Momentum, RMSProp and Adam | 3 Marks

A single weight starts at $w_0 = 2$. Two successive gradients are $g_1 = 4$ and $g_2 = 1$, with learning rate $\eta = 0.1$. For momentum, use $v_t = \beta v_{t-1} + g_t$ and $w_t = w_{t-1} - \eta v_t$ with $\beta = 0.9$ and $v_0 = 0$.

a) Compute w_1 and w_2 using plain SGD and then using SGD with momentum.

b) Explain which problem of plain SGD is addressed by momentum, by RMSProp and by Adam.

c) State one situation where Adam is preferred over plain SGD and one situation where plain SGD may still be preferred.

Evidence Expected: *Numerical updates for both methods, explanation of each variant and context-based choice.*

TASK 11 – Curse of Dimensionality | 3 Marks

The factory plans to extend the sensor set from 4 features to 100 features. Every feature is scaled to [0, 1] and divided into 10 equal bins. Only 5,000 labelled records are available.

a) Calculate the number of cells in the feature space for d = 4 and d = 100, and compare it with 5,000 records.

b) For uniformly distributed data, a sub-cube must contain 10% of the data. Using r = 0.1^(1/d), compute the required edge length for d = 2, 10 and 100 and interpret the result.

c) Explain the consequences for a k-nearest neighbour classifier and suggest two ways to reduce the problem.

Evidence Expected: *Cell counts, edge-length calculations, interpretation and mitigation strategies.*

TASK 12 – Design Challenge: Selecting and Adapting a Learning Strategy | 8 Marks

Changed Requirement

The factory initially analyses 5,000 records with 4 sensor features using a logistic regression model trained by full-batch gradient descent. It must now process 2,000,000 records with 500 sensor features per component, predict the defect probability in near real time, and handle correlated features. Decide whether to retain, modify or redesign the strategy.

A. Impact Analysis | 2 Marks

a) Explain how the larger dataset and 500 features affect computation, memory and the training procedure.

b) Discuss why dimensionality, feature correlation and gradient behaviour become important.

B. Evaluation of the Original Approach | 2 Marks

a) Evaluate whether a linear model trained with full-batch gradient descent is sufficient for the new requirement.

b) Explain the limitations of the original approach in terms of capacity, optimization cost and the curse of dimensionality.

C. Adaptation or Redesign | 2 Marks

a) Propose a redesigned strategy using a Multilayer Perceptron trained with back-propagation and a suitable SGD variant.

b) Explain how maximum likelihood (the loss function), regularization, dimensionality reduction and the learning rate can be incorporated.

D. Evidence-Based Defence | 2 Marks

a) Propose experiments and evaluation measures to compare the redesigned approach with the original one.

b) Justify the final design using mathematical reasoning, generalization, computational needs and data characteristics.

Evidence Expected: *Analysis of data scale and dimensionality, justified learning strategy, mathematical reasoning and a suitable evaluation plan.*

Final Mark Distribution – CO1

Task	Topics Covered	Marks
Task 1	Learning Algorithms – T, E, P and Paradigm	4
Task 2	Capacity, Overfitting and Generalization	3
Task 3	Maximum Likelihood Estimation – Bernoulli	4
Task 4	Maximum Likelihood Estimation – Gaussian and MSE	5
Task 5	Building a Machine Learning Algorithm	5
Task 6	Multilayer Perceptron – Forward Pass	4
Task 7	Back-propagation Algorithm	5
Task 8	Vanishing Gradient and Activation Functions	3
Task 9	Batch, Stochastic and Mini-batch Gradient Descent	3
Task 10	SGD Variants – Momentum, RMSProp, Adam	3
Task 11	Curse of Dimensionality	3
Sub Total		**42**
Task 12	Design Challenge – Selecting and Adapting a Learning Strategy	8
TOTAL		**50**

Code of Conduct:

*I **Karthik V - 192425071** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Karthik V

ANSWERS TO ALL QUESTIONS

TASK 1 – Learning Algorithms: Task, Experience and Performance | 4 Marks

Answer 1(a): Following Mitchell's definition: a program learns from experience E with respect to task T and performance measure P if its performance at T, measured by P, improves with E.

- **Task (T):** Binary classification – given the four sensor readings x = (temperature, vibration, pressure, humidity) of a new component, predict the label y ∈ {defective, acceptable}, i.e. learn f: $\mathbb{R}^4 \rightarrow \{0, 1\}$.
- **Experience (E):** The 5,000 labelled inspection records $\{(x_i, y_i)\}$, i = 1…5000, where each x_i has four sensor values and y_i is the inspector's label.
- **Performance (P):** How correctly the system labels unseen (held-out) components – measured by recall on defective parts, precision, F1-score (and overall accuracy / cost-weighted error) on a test set that was not used for training.

Answer 1(b): Paradigm: Supervised learning (binary classification), because every training example has a known target label.

Why a fixed hand-written rule is less suitable:

- A rule such as "if vibration > a AND temperature > b then defective" needs thresholds that an expert must guess and tune by hand; with four interacting sensors the number of possible rule combinations is very large.
- The boundary between defective and acceptable is usually non-linear and noisy; a hand-written rule cannot be optimised against data, whereas a learning algorithm minimises a loss on the 5,000 records and adapts its parameters automatically.
- Rules are brittle – when machines, materials or sensor calibration change, all thresholds must be rewritten; a learned model is simply retrained on new labelled data.
- A learning algorithm can be validated on unseen data to estimate how it will generalize; a hand-written rule has no such built-in guarantee.

Answer 1(c): With only 3% defective components the classes are highly imbalanced. A useless model that always predicts "acceptable" gets Accuracy = 97% (for 5,000 records, 4,850 correct) but detects **0 of the 150 defective parts** (recall = 0). High accuracy therefore hides the failure on the class that matters most, and a missed defect (false negative) is usually much costlier than a false alarm.

Two better measures:

- **Recall (sensitivity) = TP / (TP + FN)** – the fraction of truly defective components that are caught.
- **Precision = TP / (TP + FP)** and the **F1-score = 2·Precision·Recall / (Precision + Recall)** – precision shows how many alarms are real, F1 balances both. (A confusion matrix or PR-AUC may also be reported.)

TASK 2 – Capacity, Overfitting and Generalization | 3 Marks

Answer 2(a):

Model	Training error	Validation error	Gap (Val − Train)	Diagnosis
X	18%	20%	2%	Underfits (high bias)
Y	2%	19%	17%	Overfits (high variance)
Z	6%	8%	2%	Generalizes best

Model X underfits – both training and validation errors are high and close together, so it is too simple to capture the pattern. **Model Y overfits** – very low training error but a validation error almost 10× larger, meaning it memorised noise. **Model Z generalizes best** – lowest validation error (8%) with a small gap of only 2%.

Answer 2(b): Capacity is the range of functions a model can represent (number of parameters, hidden units/layers, polynomial degree). The generalization gap is **validation (test) error − training error**.

- **Low capacity:** the model cannot even fit the training data → high training error (underfitting); the gap is small but the total error is high (Model X).
- **Capacity increases:** training error keeps falling, but the model starts fitting noise, so validation error eventually rises and the generalization gap widens (Model Y).
- Plotting error against capacity gives a U-shaped validation curve; the best model has **optimal capacity** at the bottom of the U, where the bias–variance trade-off is balanced (Model Z). Generalization error ≈ training error + generalization gap.

Answer 2(c):

- **Model X (underfitting):** increase capacity – add hidden units/layers or non-linear (polynomial) features, reduce regularisation, or train longer – so that training error falls.
- **Model Y (overfitting):** add regularisation (L2 weight decay or dropout) or early stopping, and/or collect more training data or augment it, or reduce model size – so that the generalization gap shrinks.

TASK 3 – Maximum Likelihood Estimation: Bernoulli Model | 4 Marks

Answer 3(a): Let $x_i = 1$ if component i is defective, 0 otherwise; $P(x_i) = p^{x_i} (1 - p)^{(1 - x_i)}$. With independent components, n = 50 and $k = \Sigma x_i = 7$:

$$L(p) = \prod p^{x_i} (1 - p)^{(1-x_i)} = p^7 (1 - p)^{43}$$

$$\ell(p) = \ln L(p) = 7 \ln p + 43 \ln(1 - p)$$

Answer 3(b):

$$d\ell/dp = 7/p - 43/(1 - p) = 0 \implies 7(1 - p) = 43p \implies 7 = 50p$$

$$\hat{p} = k / n = 7 / 50 = 0.14$$

Second derivative: $d^2\ell/dp^2 = -7/p^2 - 43/(1 - p)^2 < 0$, so this is a maximum. **MLE: $\hat{p} = 0.14$ (14% defective).**

Answer 3(c):

$$P(\text{three acceptable}) = (1 - \hat{p})^3 = (0.86)^3 = 0.636 \quad (\approx 63.6\%)$$

Reliability for a small sample: the MLE is consistent, but with n = 50 it has considerable variance. Standard error = $\sqrt{(\hat{p}(1 - \hat{p})/n)} = \sqrt{(0.14 \times 0.86 / 50)} \approx 0.049$, giving an approximate 95% interval $p \in (0.044, 0.236)$. Over this range, the answer $(1 - p)^3$ varies from about 0.45 to 0.87, so the estimate of 0.636 is quite uncertain. A larger sample (or a prior, as in Bayesian/MAP estimation) is needed for a reliable value; with a very small sample the MLE can even give extreme values such as $\hat{p} = 0$.

TASK 4 – Maximum Likelihood Estimation: Gaussian Model and Link to MSE | 5 Marks

Answer 4(a):

$$L(\mu, \sigma^2) = \prod_i (2\pi\sigma^2)^{(-1/2)} \exp[\, -(x_i - \mu)^2 / (2\sigma^2) \,]$$

$$\ell(\mu, \sigma^2) = -(n/2) \ln(2\pi) - (n/2) \ln \sigma^2 - (1/(2\sigma^2)) \Sigma_i (x_i - \mu)^2$$

Answer 4(b): For μ: $\partial\ell/\partial\mu = (1/\sigma^2) \Sigma(x_i - \mu) = 0 \implies \mathbf{\hat{\mu} = (1/n) \Sigma x_i = \bar{x}}$.

For σ²: $\partial\ell/\partial\sigma^2 = -n/(2\sigma^2) + \Sigma(x_i - \mu)^2/(2\sigma^4) = 0 \implies \mathbf{\hat{\sigma}^2 = (1/n) \Sigma (x_i - \hat{\mu})^2}$.

Values: $\hat{\mu} = (68 + 70 + 72 + 74 + 76)/5 = 360/5 =$ **72 °C**. Deviations: −4, −2, 0, 2, 4; squared: 16, 4, 0, 4, 16; sum = 40. $\hat{\sigma}^2 = 40/5 =$ **8 °C²** ($\hat{\sigma} \approx 2.83$ °C).

Answer 4(c): The MLE uses the sample mean $\bar{x}$ instead of the true μ. Because $\Sigma(x_i - \mu)^2 = \Sigma(x_i - \bar{x})^2 + n(\bar{x} - \mu)^2$, taking expectations gives $n\sigma^2 = E[\Sigma(x_i - \bar{x})^2] + \sigma^2$, so $E[\Sigma(x_i - \bar{x})^2] = (n - 1)\sigma^2$. Hence

$$E[\hat{\sigma}^2] = ((n - 1)/n)\, \sigma^2 < \sigma^2$$

i.e. the MLE systematically **underestimates** the variance (deviations from the sample mean are smaller than from the true mean).

Unbiased estimate: $s^2 = (1/(n - 1)) \Sigma(x_i - \bar{x})^2 = 40/4 =$ **10 °C²**. **Comparison:** 8 (MLE) versus 10 (unbiased); the MLE is smaller by the factor $(n - 1)/n = 4/5 = 0.8$, i.e. 20% lower. For large n the two values converge, so the MLE is still consistent.

Answer 4(d): Given $x, y \sim N(wx + b, \sigma^2)$, so for m data points:

$$L(w, b) = \prod_i (2\pi\sigma^2)^{(-1/2)} \exp[\, -(y_i - wx_i - b)^2 / (2\sigma^2) \,]$$

$$\ell(w, b) = -(m/2) \ln(2\pi\sigma^2) - (1/(2\sigma^2)) \Sigma_i (y_i - wx_i - b)^2$$

The first term and the factor $1/(2\sigma^2)$ do not depend on w or b (σ^2 is a positive constant). Therefore

$$\text{argmax}_{w,b}\ \ell(w, b) = \text{argmin}_{w,b} \Sigma_i (y_i - \hat{y}_i)^2 = \text{argmin}_{w,b} (1/m) \Sigma_i (y_i - \hat{y}_i)^2 = \text{argmin MSE}$$

Maximizing the Gaussian likelihood is equivalent to minimizing the mean squared error: **least squares is maximum likelihood under Gaussian noise.**

TASK 5 – Building a Machine Learning Algorithm | 5 Marks

Answer 5(a):

- **Dataset:** D = {(1, 3), (2, 5), (3, 6), (4, 9)}, m = 4 (x = running time in hours, y = tool wear in 0.1 mm).
- **Model:** $\hat{y} = wx + b$ with parameters w and b.
- **Cost function:** $J(w, b) = (1/m) \Sigma (\hat{y}_i - y_i)^2$ (mean squared error).
- **Optimization procedure:** gradient descent: $w \leftarrow w - \alpha\, \partial J/\partial w$, $b \leftarrow b - \alpha\, \partial J/\partial b$.

Answer 5(b): With $e_i = \hat{y}_i - y_i = wx_i + b - y_i$, $J = (1/m) \Sigma e_i^2$. By the chain rule, $\partial J/\partial w = (1/m) \Sigma 2e_i \cdot \partial e_i/\partial w$, and $\partial e_i/\partial w = x_i$, $\partial e_i/\partial b = 1$. Hence

$$\partial J/\partial w = (2/m) \Sigma_i (\hat{y}_i - y_i)\, x_i \qquad \partial J/\partial b = (2/m) \Sigma_i (\hat{y}_i - y_i)$$

Answer 5(c): At $w = b = 0$, $\hat{y} = 0$ for all points, so errors $e_i = \hat{y} - y = -3, -5, -6, -9$ ($J = (9 + 25 + 36 + 81)/4 =$ **37.75**).

$\Sigma e_i = -23$; $\Sigma e_i x_i = (-3)(1) + (-5)(2) + (-6)(3) + (-9)(4) = -3 - 10 - 18 - 36 = -67$.

$$\partial J/\partial w = (2/4)(-67) = -33.5 \qquad \partial J/\partial b = (2/4)(-23) = -11.5$$

$$w_new = 0 - 0.05(-33.5) = 1.675 \qquad b_new = 0 - 0.05(-11.5) = 0.575$$

New w = 1.675 and b = 0.575. Check: the new predictions are 2.25, 3.925, 5.6, 7.275, so J falls from 37.75 to ≈ 1.21.

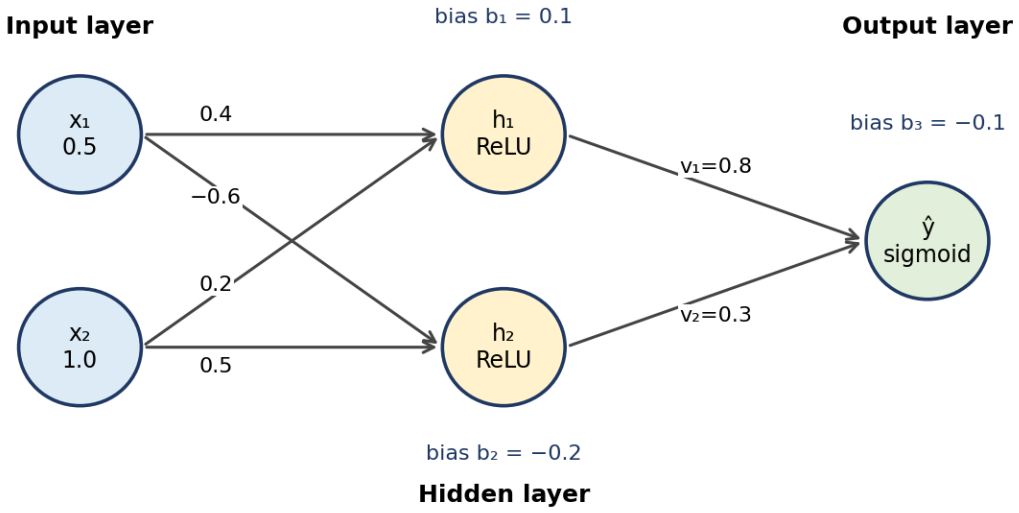
Answer 5(d):

- **Parameters** (w, b): values learned from the data by gradient descent; they define the fitted line.
- **Hyperparameters:** values set before/outside training and not learned by the cost – learning rate $\alpha = 0.05$, number of iterations/epochs, the initial values $w = b = 0$, the batch size (full batch here) and the choice of a linear model.

Effect of α: if α is **too large**, the updates overshoot the minimum, the cost oscillates or even diverges. For this problem the Hessian of J is [[15, 5], [5, 2]] with largest eigenvalue ≈ 16.7, so gradient descent is stable only for $\alpha < 2/16.7 \approx 0.12$; $\alpha = 0.05$ is safe. If α is **too small**, convergence is very slow (many iterations; the flat direction with eigenvalue ≈ 0.3 needs thousands of steps) and training may appear to stall.

TASK 6 – Multilayer Perceptron: Architecture and Forward Pass | 4 Marks

Answer 6(a):



Trainable parameters: hidden layer 2 neurons × (2 weights + 1 bias) = 6; output layer 2 weights + 1 bias = 3.

$$\text{Total} = (2\times2 + 2) + (2\times1 + 1) = 6 + 3 = 9 \text{ trainable parameters}$$

Answer 6(b):

$$z_1 = 0.4(0.5) + 0.2(1.0) + 0.1 = 0.5 \implies h_1 = \text{ReLU}(0.5) = 0.5$$

$$z_2 = -0.6(0.5) + 0.5(1.0) - 0.2 = 0 \implies h_2 = \text{ReLU}(0) = 0$$

$$z_o = 0.8(0.5) + 0.3(0) - 0.1 = 0.3 \implies \hat{y} = \sigma(0.3) = 1/(1 + e\text{^}(-0.3)) = 0.5744$$

Result: $h_1 = 0.5$, $h_2 = 0$, $\hat{y} \approx 0.574$ (a 57.4% predicted probability of the positive class).

Answer 6(c): Without a non-linearity, stacked layers collapse into one linear map: $W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$, which is again affine. **If all activations are linear**, the network – however many layers it has – is equivalent to a single linear model (with a sigmoid output, just logistic regression) and can only form a linear decision boundary; it cannot solve problems such as XOR or curved defect regions caused by interactions between sensors.

A non-linear hidden activation (e.g. ReLU) lets the layers bend and combine features, so the MLP can approximate any continuous function (universal approximation) and learn complex, non-linear boundaries.

TASK 7 – Back-propagation Algorithm | 5 Marks

Answer 7(a): Notation: $z_o = v_1h_1 + v_2h_2 + b_o$, $\hat{y} = \sigma(z_o)$, $L = \frac{1}{2}(t - \hat{y})^2$, $h_1 = \text{ReLU}(z_1)$, $z_1 = w_{11}x_1 + w_{12}x_2 + b_1$.

$$\partial L/\partial v_1 = (\partial L/\partial \hat{y})(\partial \hat{y}/\partial z_o)(\partial z_o/\partial v_1) = (\hat{y} - t) \cdot \hat{y}(1 - \hat{y}) \cdot h_1$$

$$\partial L/\partial w_{11} = (\partial L/\partial \hat{y})(\partial \hat{y}/\partial z_o)(\partial z_o/\partial h_1)(\partial h_1/\partial z_1)(\partial z_1/\partial w_{11}) = (\hat{y} - t) \cdot \hat{y}(1 - \hat{y}) \cdot v_1 \cdot 1[z_1 > 0] \cdot x_1$$

Answer 7(b): From Task 6: $\hat{y} = 0.5744$, $h_1 = 0.5$, $h_2 = 0$, $z_1 = 0.5 > 0$, $z_2 = 0$, $x_1 = 0.5$.

$$\partial L/\partial \hat{y} = \hat{y} - t = -0.4256; \quad \sigma' = \hat{y}(1 - \hat{y}) = 0.5744 \times 0.4256 = 0.2445$$

$$\delta_o = \partial L/\partial z_o = -0.4256 \times 0.2445 = -0.1040$$

$$\partial L/\partial v_1 = \delta_o \cdot h_1 = -0.1040 \times 0.5 = -0.0520$$

$$\partial L/\partial v_2 = \delta_o \cdot h_2 = -0.1040 \times 0 = 0$$

$$\partial L/\partial b_o = \delta_o = -0.1040$$

$$\partial L/\partial w_{11} = \delta_o \cdot v_1 \cdot 1[z_1 > 0] \cdot x_1 = -0.1040 \times 0.8 \times 1 \times 0.5 = -0.0416$$

(The ReLU derivative at $z_2 = 0$ is taken as 0, the usual convention.)

Answer 7(c):

$$v_1 \leftarrow v_1 - \eta\, \partial L/\partial v_1 = 0.8 - 0.1(-0.0520) = 0.8052$$

$$w_{11} \leftarrow w_{11} - \eta\, \partial L/\partial w_{11} = 0.4 - 0.1(-0.0416) = 0.4042$$

Check: with these updates $\hat{y}$ rises from 0.5744 to ≈ 0.5755 and the loss falls from ≈ 0.0905 to ≈ 0.0901.

Answer 7(d): Computing every derivative independently (e.g. by numerical differentiation or by expanding each chain rule from scratch) needs a separate forward pass per parameter, costing $O(P^2)$ for P parameters, and repeats the same sub-expressions many times. **Back-propagation** is dynamic programming on the computational graph: one forward pass stores the activations, then one backward pass computes the shared error terms (such as δ_o above, which is reused for v_1, v_2, b_o and propagated to every hidden weight) once and reuses them. All P gradients are obtained at a cost of only about 2–3 times one forward pass, i.e. O(P). This is what makes training networks with millions of parameters feasible.

TASK 8 – Vanishing Gradient and Activation Functions | 3 Marks

Answer 8(a): $\sigma(z) = 1/(1 + e^{-z})$ and $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. Let $s = \sigma(z) \in (0, 1)$; $f(s) = s(1 - s)$ has $f'(s) = 1 - 2s = 0$ at $s = 0.5$ and $f'' < 0$, so its maximum is $f(0.5) = 0.25$, reached at $z = 0$. Hence **$\sigma'(z) \leq 0.25$ for all z**.

$$0.25^{10} = 1/4^{10} = 1/1{,}048{,}576 \approx 9.54 \times 10^{-7}$$

Answer 8(b): In back-propagation the gradient reaching layer 1 contains the product of the activation derivatives (and weights) of all layers above it: $\partial L/\partial W^{(1)} \propto \prod \sigma'(z^{(l)}) \cdot W^{(l)} \ldots$. With ten sigmoid layers this product is at most about 10^{-6} times the gradient at the output (unless the weights are very large), and $\sigma' \approx 0$ in the saturated regions makes it even smaller. The gradient therefore shrinks exponentially with depth.

Effect on learning: the weight updates of the early layers are almost zero, so these layers learn extremely slowly or not at all; the training loss barely decreases, the network cannot build good low-level features, and the later layers work on poor inputs.

Answer 8(c):

- **Use ReLU / Leaky ReLU (or GELU) activations:** for positive inputs the derivative is exactly 1, so the product of activation derivatives does not shrink with depth; ReLU is also cheap to compute. (Leaky ReLU avoids "dead" neurons.)
- **Batch normalization with He/Xavier initialization (or residual/skip connections):** normalization keeps pre-activations in a well-scaled, non-saturated range and initialization keeps the variance of the gradients constant across layers; skip connections give the gradient a direct identity path to early layers.

TASK 9 – Batch, Stochastic and Mini-batch Gradient Descent | 3 Marks

Answer 9(a):

- **Batch gradient descent:** 1 update per epoch (one gradient over all N).
- **SGD (batch size 1):** N = 1,000,000 updates per epoch.
- **Mini-batch (B = 100):** N/B = 1,000,000/100 = **10,000 updates** per epoch.

Answer 9(b):

Aspect	Batch GD	SGD	Mini-batch (B = 100)
Gradient noise	None – exact gradient	Very high (single sample)	Moderate (variance falls by ≈ 1/B)
Memory use	Highest – all 10^6 records per step	Lowest – one record	Low – 100 records fit in memory/GPU
Cost per update	Highest: O(N·P)	Lowest: O(P), but poorly vectorised	Medium: O(100·P), well vectorised
Convergence	Smooth and stable, but only 1 step per epoch → slow; can stall at saddles	Fast initial progress, oscillates around the minimum, needs decaying learning rate	Fast and fairly stable; noise helps escape poor minima

Answer 9(c): Recommendation: mini-batch gradient descent (batch size about 100–256) with a momentum/Adam optimizer. It gives 10,000 updates per epoch (10^4 times more than batch GD), each

costing only 1/10,000 of a full-batch gradient; the gradient noise is reduced by about 10× compared with pure SGD; the batch fits easily in memory and uses vectorized/GPU hardware efficiently; and the mild noise helps generalization. Batch GD is too slow and memory-heavy for 10^6 records, and pure SGD is too noisy and hardware-inefficient.

TASK 10 – SGD Variants: Momentum, RMSProp and Adam | 3 Marks

Answer 10(a): Plain SGD: $w_1 = 2 - 0.1(4) = \mathbf{1.6}$; $w_2 = 1.6 - 0.1(1) = \mathbf{1.5}$.

SGD with momentum ($\beta = 0.9$, $v_0 = 0$):

$$v_1 = 0.9(0) + 4 = 4 \implies w_1 = 2 - 0.1(4) = 1.6$$

$$v_2 = 0.9(4) + 1 = 4.6 \implies w_2 = 1.6 - 0.1(4.6) = 1.14$$

Momentum travels further ($w_2 = 1.14$ vs 1.5) because the velocity retains the earlier large gradient.

Answer 10(b):

- **Momentum:** addresses slow progress and zig-zag oscillation of plain SGD in ravines/narrow valleys and on plateaus, and noisy gradients. By accumulating an exponentially-weighted sum of past gradients, it cancels oscillating components and speeds up consistent directions.
- **RMSProp:** addresses the use of one single learning rate for all parameters when gradient magnitudes differ widely. It divides each step by a running RMS of that parameter's gradient, $s_t = \rho\, s_{(t-1)} + (1-\rho)g^2$, $w \leftarrow w - \eta\, g/(\sqrt{s_t} + \varepsilon)$, giving per-parameter adaptive step sizes (and avoiding AdaGrad's ever-shrinking rate).
- **Adam:** combines both – a momentum-like first-moment estimate m_t and an RMSProp-like second-moment estimate v_t with bias correction, $w \leftarrow w - \eta\, \hat{m}_t/(\sqrt{\hat{v}_t} + \varepsilon)$ – so it handles oscillation and scale differences together and works well with little tuning.

Answer 10(c):

- **Adam preferred:** training deep or sparse-feature networks with noisy gradients and parameters of very different gradient scales (e.g. a 500-feature MLP, or NLP models), or when fast convergence with minimal learning-rate tuning is needed.
- **Plain SGD (with momentum) may still be preferred:** when the best final generalization is needed on a well-tuned large problem (e.g. image CNNs, where Adam may settle in sharper minima), when memory is tight (SGD stores no extra moment vectors), or for simple convex problems such as logistic regression.

TASK 11 – Curse of Dimensionality | 3 Marks

Answer 11(a):

- **d = 4:** $10^4 = 10{,}000$ cells; 5,000 records $\implies$ only 0.5 record per cell on average, so at least half of the cells are empty even for four features.
- **d = 100:** 10^{100} cells (more than the number of atoms in the universe, $\approx 10^{80}$); 5,000 records can occupy at most $5{,}000/10^{100} = 5 \times 10^{-97}$ of them – the space is almost entirely empty.

Answer 11(b):

d = 2: r = 0.1^(1/2) = 0.316 d = 10: r = 0.1^(1/10) = 0.794 d = 100: r = 0.1^(1/100) = 0.977

Interpretation: to capture just 10% of the data, the sub-cube must cover 31.6% of each axis in 2-D, 79.4% in 10-D and 97.7% of the range of every feature in 100-D. A "local" neighbourhood is therefore no longer local – it spans almost the whole space – so local methods lose their meaning.

Answer 11(c): Consequences for k-NN: in high dimensions the points are sparse and the distances between a query and its nearest and farthest neighbours become almost equal (distance concentration), so the "nearest" neighbours are not truly similar; irrelevant or noisy features dominate the Euclidean distance; many more samples are needed (exponential in d) to get dense coverage; prediction cost $O(N \cdot d)$ grows; and accuracy drops with overfitting to noise.

Two ways to reduce the problem:

- **Dimensionality reduction / feature extraction** (PCA, autoencoders) to project the 100 features onto a few informative components.
- **Feature selection** (remove irrelevant or highly correlated sensors, using mutual information or L1 regularisation), and/or collecting more labelled data.

TASK 12 – Design Challenge: Selecting and Adapting a Learning Strategy | 8 Marks

Answer 12A(a):

- **Computation:** a full-batch gradient touches $N \times d = 2{,}000{,}000 \times 500 = 10^9$ values and $\approx 10^9$ multiply-adds per iteration; hundreds or thousands of iterations ($\approx 10^{12}$) make retraining very slow, while the original data ($5{,}000 \times 4 = 20{,}000$ values) was trivial.
- **Memory:** the design matrix alone is 10^9 numbers $\approx$ 4 GB in float32 (8 GB in float64), plus activations and gradients – it may not fit into memory at once, so the data must be streamed in mini-batches.
- **Training procedure:** one update per pass over the data is no longer practical; training must move to mini-batch/stochastic gradient descent (e.g. 512 samples $\rightarrow \approx$ 3,900 updates per epoch), with data pipelines, feature scaling and a validation split, and predictions must be fast enough for near-real-time use.

Answer 12A(b):

- **Dimensionality:** with 500 features the feature space is astronomically large (10^{500} cells), so even 2 million records are sparse; models with many parameters overfit easily, and distances lose meaning (curse of dimensionality) – regularisation and dimensionality reduction become essential.
- **Feature correlation:** correlated features make X^TX nearly singular and the loss surface ill-conditioned (large ratio $\lambda max/\lambda min$), so gradient descent zig-zags in narrow valleys, needs a small α, converges slowly and gives unstable, high-variance weights; redundant features waste computation.
- **Gradient behaviour:** the stable step size is limited by $\alpha < 2/\lambda max$, features on different scales produce gradients of different magnitudes, mini-batch gradients are noisy, and in a deeper network gradients can vanish or explode – so normalization, careful initialization and adaptive optimizers matter.

Answer 12B(a): Not sufficient. Logistic regression can only produce a linear decision boundary (a hyperplane) in the 500-dimensional space, while real defects usually result from non-linear interactions between sensors (e.g. high temperature combined with high vibration). Full-batch gradient descent is also too slow and memory-intensive for $2 \times 10^6 \times 500$ data and cannot support frequent retraining. Logistic regression is still useful as a fast, interpretable **baseline** and its probabilistic (Bernoulli/MLE) formulation should be retained, but the training strategy and model capacity must change.

Answer 12B(b):

- **Capacity:** a linear model has only $d + 1 = 501$ parameters and high bias – it underfits non-linear patterns however much data is added.
- **Optimization cost:** each full-batch iteration costs $O(N \cdot d) \approx 10^9$ operations and gives just one update; correlated features make the problem ill-conditioned, so many iterations are required.
- **Curse of dimensionality:** in 500-D the data are sparse, many weights are poorly determined and noise features add variance; without regularisation or feature reduction the model overfits, and a linear model cannot exploit the lower-dimensional structure of correlated sensors.

Answer 12C(a): Decision: redesign (keep the probabilistic principle, change model and optimizer). Pipeline:

- Standardise each sensor feature (z-score) and optionally apply PCA to keep $\approx$ 95–99% of the variance ($500 \rightarrow \approx 100$ inputs).
- **MLP:** input $\rightarrow$ Dense(128, ReLU) $\rightarrow$ Dense(64, ReLU) $\rightarrow$ Dense(1, sigmoid), He initialization. For 500 inputs the parameter count is $(500 \times 128 + 128) + (128 \times 64 + 64) + (64 + 1) = 72{,}449$ ($\approx$ 21,000 if PCA reduces the input to 100).

- **Training:** back-propagation with mini-batches (e.g. 512 → ≈ 3,900 updates per epoch) and the **Adam** optimizer ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\eta = 10^{-3}$), run on streamed data. Inference needs only ≈ 72,000 multiply-adds per component, which meets the near-real-time requirement.

Answer 12C(b):

- **Maximum likelihood / loss:** treat the label as Bernoulli($\hat{y}$); maximizing the likelihood equals minimizing the binary cross-entropy $J = -(1/m)\,\Sigma[y \ln \hat{y} + (1 - y) \ln(1 - \hat{y})]$ (class weights can be added for the 3% imbalance). Its output-layer gradient is simply $\hat{y} - y$, which avoids the slow learning that MSE with a sigmoid would cause.
- **Regularization:** L2 weight decay $\lambda\|W\|^2$ (equal to a Gaussian prior, i.e. MAP estimation), dropout (0.2–0.5) and early stopping on the validation loss to keep the generalization gap small.
- **Dimensionality reduction:** standardisation + PCA (or an autoencoder / feature selection) removes redundant correlated features, reducing parameters and improving conditioning.
- **Learning rate:** choose η by a validation search/learning-rate sweep, start at ≈ 10^{-3} for Adam and decay it (step or cosine schedule) so that the noise of mini-batch training dies down near the minimum.

Answer 12D(a):

- **Data protocol:** one chronological split of the 2,000,000 records – 70% train, 15% validation, 15% test – used identically for both approaches; repeat with several random seeds (mean ± standard deviation, paired bootstrap or significance test).
- **Compared systems:** (1) original: logistic regression with full-batch GD; (2) logistic regression with mini-batch SGD/Adam (to separate the effect of the optimizer); (3) proposed MLP + Adam, with ablations: with/without PCA, with/without L2/dropout, SGD vs momentum vs Adam.
- **Predictive measures:** precision, recall, F1-score, PR-AUC and ROC-AUC, confusion matrix, log-loss/calibration, with the decision threshold chosen using the cost of a missed defect.
- **Generalization and optimization:** learning curves (training vs validation loss per epoch), generalization gap, convergence speed (epochs/time to a target loss).
- **Computational measures:** training time and memory per epoch, and inference latency per component (e.g. target < 10 ms) and throughput.

Answer 12D(b):

- **Mathematical reasoning:** binary cross-entropy is the Bernoulli MLE (as in Task 3), giving well-calibrated probabilities; the MLP removes the linear-boundary bias (universal approximation); back-propagation computes all gradients in O(P); mini-batch updates give ≈ 3,900 steps per epoch versus 1 for full-batch with gradient variance falling as 1/B; Adam rescales each parameter so ill-conditioning caused by correlated/mis-scaled features hurts less; PCA lowers d and improves the condition number.
- **Generalization:** L2, dropout and early stopping control variance, so the validation − training gap stays small, while the higher capacity lowers the bias (underfitting of Model X type).
- **Data characteristics:** 2×10^6 records for ≈ 72,000 parameters is ≈ 27 samples per parameter, enough to train a moderately sized MLP without severe overfitting, and dimensionality reduction counters the sparsity of the 500-D space.
- **Computational needs:** one epoch costs ≈ $2 \times 10^6 \times 72{,}449 \times 6 \approx 9 \times 10^{11}$ FLOPs (forward + backward) – seconds to minutes on a GPU – with only mini-batches in memory, and inference of ≈ 72,000 multiply-adds per component easily meets the near-real-time requirement.

Conclusion: logistic regression stays as the baseline, but the strategy is redesigned to an MLP trained by mini-batch back-propagation with Adam, a cross-entropy (MLE) loss, regularisation and dimensionality reduction; the experiments above provide the evidence to confirm this choice.